

COMP90077

Lecture 3

Submodular Maximization

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Today

- Greedy Approximation Algorithms
- Submodular Maximization
- Assignment 1
- Tips

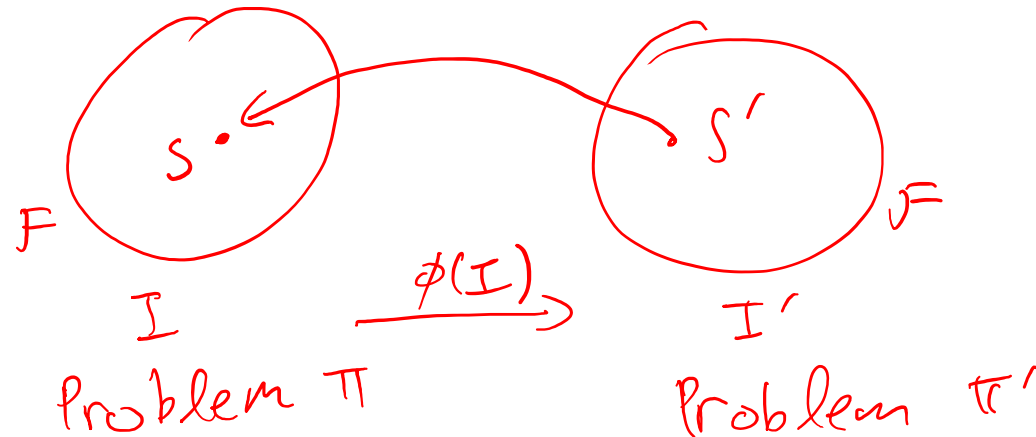
Problems and Instances

- Need to more precisely define problems and instances
- Def. A *combinatorial optimization problem* Π is a collection of instances I . An *instance* consists of:
 - Finite set of elements E
 - **(Objective function)** g that maps subsets of E to real numbers
 - **(Feasible sets/solutions)** Collection $\mathcal{F}$ of subsets of E
 - **Goal:** Find a feasible subset S that maximizes/minimizes g
 - Use $OPT(I)$ to denote the maximum/minimum objective value
- Def. An efficient exact algorithm for Π is one that on every instance I , - outputs a feasible solution S with $g(S) = OPT(I)$ in time polynomial in representation of I

Reductions

Def. Problem Π reduces to problem Π' if for every instance I of Π ,

- can efficiently compute an instance $\phi(I)$ such that
- For every optimal solution S' of $\phi(I)$, can efficiently compute an optimal solution S of I



Approximation Algorithms

- Def. An efficient exact algorithm for Π is one that on every instance I , - outputs a feasible solution S with $g(S) = OPT(I)$ in time polynomial in representation of I
- Many problems are NP-hard
- Widely believed not to have efficient exact algorithms
- Def. A *c-approximation algorithm* for Π is one that on every instance I :
 - runs in polynomial time
 - (maximization) outputs a feasible solution S with $g(S) \geq c \cdot OPT(I)$ ($c \leq 1$)
 - (minimization) outputs a feasible solution S with $g(S) \leq c \cdot OPT(I)$ ($c \geq 1$)

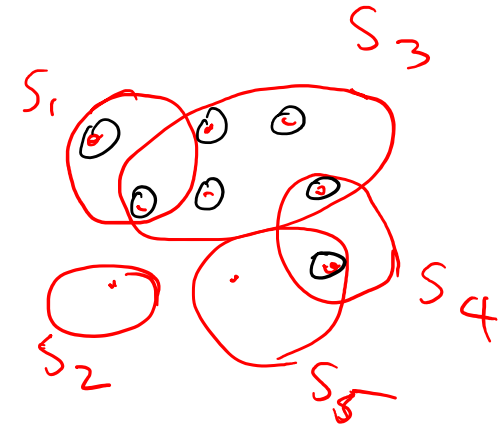
Set Systems and Coverage

Def. A *set system* consists of

- A *universe* U of n elements
- A collection of m sets $S_1, \dots, S_m$

Def. Given $X \subseteq [m] = \{1, \dots, m\}$, the *coverage* of X is the number of elements covered by X

$$\text{cov}(X) = \left| \bigcup_{j \in X} S_j \right|$$



$$X = \{1, 3, 4\}.$$
$$\text{cov}(X) = 7$$

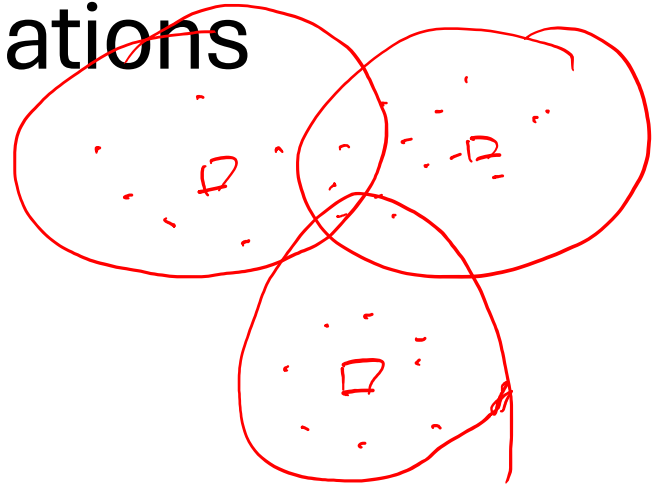
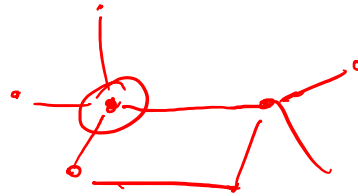
Coverage Maximization Problem

Def. An instance of the Coverage Maximization Problem consists of:

- A set system with n elements and m sets $S_1, \dots, S_m$.
- A bound k .
- Goal: find k sets with maximum coverage
- Equivalently, find $I \subseteq [m]$ with $|I| \leq k$ that maximizes $cov(I)$

Coverage Maximization Applications

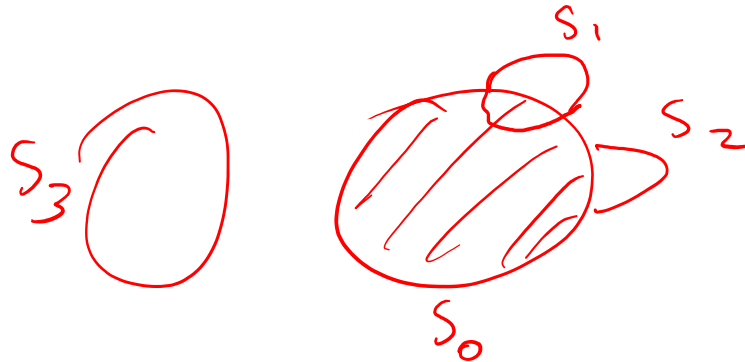
- Communications tower placement
- Guard placement
- Influence maximization



Coverage Maximization Problem

Def. An instance of the Coverage Maximization Problem consists of:

- A set system with n elements and m sets $S_1, \dots, S_m$.
- A bound k .
- Goal: find k sets with maximum coverage
- Equivalently, find $I \subseteq [m]$ with $|I| \leq k$ that maximizes $\text{cov}(I)$
- Greedy algorithm?



Greedy Coverage Maximization

- Initialize $I \leftarrow \emptyset$
- While $|I| < k$:
 - Find $\underline{j^*}$ that maximizes increase in coverage: $\underbrace{cov(I \cup \{j^*\}) - cov(I)}$
 - Add j^* to I
- Return I

Greedy Analysis

Coverage Maximization Summary

- Greedy achieves a $(1 - 1/e)$ -approximation
- For every constant $\epsilon > 0$, NP-hard to get $(1 - 1/e + \epsilon)$ -approximation.
Implies NP-hard to get $(2/3)$ -approximation
- See why next lecture
- Next: Can we generalize?

$$\max_{S \subseteq F} g(S)$$

Submodular Functions

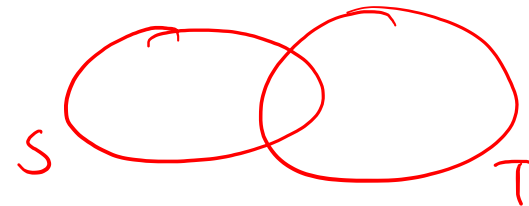
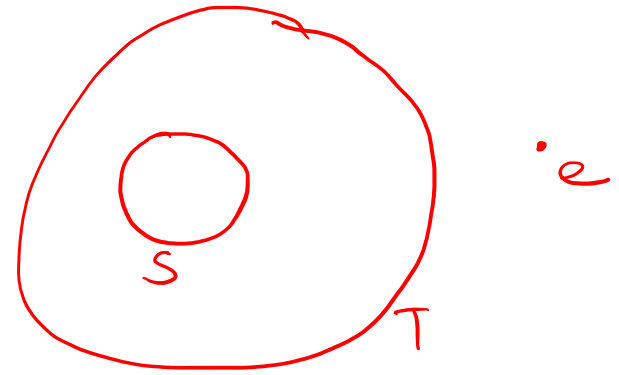
A function $g: 2^E \rightarrow R$ is *submodular* if

(Diminishing Marginals) For every $S \subseteq T \subseteq E$ and $e \notin T$,

$$\underbrace{g(S \cup \{e\}) - g(S)} \geq \underbrace{g(T \cup \{e\}) - g(T)}$$

(Definition 2) For every $S, T \subseteq E$,

$$g(S) + g(T) \geq g(S \cup T) + g(S \cap T)$$



Submodular Functions are Everywhere

- Coverage function
- Matroid rank function
- Purchasing cost with buy-at-bulk discounts
- Utility from a set of substitute goods

$g(S) = f(|S|)$  concave

Cardinality-Constrained Submod. Max.

Def. An instance consists of:

- A non-negative monotone submodular function $g: 2^E \rightarrow R$
- A bound k .
- Goal: find $I \subseteq E$ with $|I| \leq k$ that maximizes $g(I)$
- Greedy algorithm?

$$\begin{array}{l} \textcircled{O_S} \quad T \\ S \subseteq T \\ g(S) \leq g(T) \end{array}$$

Maximize marginal gain

Greedy Submodular Maximization

- Initialize $I \leftarrow \emptyset$
- While $|I| < k$:
 - Find j^* that maximizes increase in objective: $\underbrace{g(I \cup \{j^*\})}_{\text{blue}} - \underbrace{g(I)}_{\text{blue}}$
 - Add j^* to I
- Return I

Greedy Analysis

Reductions

Def. Problem Π reduces to problem Π' if for every instance I of Π ,

- can efficiently compute an instance $\phi(I)$ such that
- For every optimal solution S' of $\phi(I)$, can efficiently compute an optimal solution S of I

- Examples

Assignment 10%

- 2 questions:
 - Q1: find the mistake in an LLM-generated algorithm and proof
 - Q2: reduce a given problem to min-weight or max-weight basis problem (similar in style to Exercises 4 and 5 in Tutorial 2)
 - No need to repeat proofs of theorems/lemmas/facts in lectures and lecture notes. Just cite
 - If you want to use anything outside of course materials, you have to give the full proof

Q1 Find the Mistake

Problem Definition.

Given:

- Nonnegative, monotone submodular function $f: 2^E \rightarrow R$
- Bound k

Goal: find the largest set $S \subseteq E$ such that $f(S) \leq k$. Largest = most number of elements

CCSM

$\max f(S)$

$s.t. |S| \leq k$

$\max |S|$

$s.t. f(S) \leq k$

2. How the Greedy Algorithm Achieves This

Let's look at the standard greedy algorithm: at each step, you add the element e that results in the smallest marginal increase in your cost, which is $f(S \cup \{e\}) - f(S)$.

If we run this greedy algorithm until we pick exactly the optimal number of elements (let's call that optimal number k), we get a beautiful and mathematically provable bicriteria approximation.

Here is the step-by-step logic:

1. Assume the optimal solution S_{opt} has size k and satisfies $f(S_{opt}) \leq B$.
2. Suppose our greedy algorithm has already picked a set S_i of size i (where $i < k$).
3. Because f is submodular, if we were to add all the remaining elements of S_{opt} to our current set S_i , the total extra cost would be at most B .
4. Since there are at most k elements left in S_{opt} to add, there must be at least *one* element among them whose individual marginal cost is at most $\frac{B}{k-i}$.
5. Because the greedy algorithm always picks the element with the *absolute minimum* marginal cost, the i -th element it adds will cost no more than $\frac{B}{k-i+1}$.

3. The Mathematical Guarantee

If we sum up the costs of picking k elements this way, the total cost of our greedy set S is bounded by:

$$f(S) \leq \sum_{j=1}^k \frac{B}{j} = B \cdot H_k$$

$$\sum_{j=1}^k \frac{1}{j} = H_k$$

Where H_k is the k -th Harmonic number, which is approximately $\ln k$.

Q2 Network Redundancy Elimination

Motivation:

Suppose you are in charge of a communications network. Each network link has a leasing cost. You want to discard links while maintaining connectivity.

Problem Definition:

- Given a graph $G = (V, E)$ with weight function w on the edges.
- Goal: Find max-weight subset of edges R such that $R \setminus S$ is a connected graph

Task: Reduce to a max-weight basis or min-weight basis problem

General Tips

- Write as you read (even copying helps)
- Write as you think
- Rubber ducking
- Draw visual representations

Problem Solving Tips

1. Understand the problem

- Write out what is the set of elements, feasible sets, objective function
- Try out some examples to get a feel for the problem

2. Pattern matching

- Intended solution follows a pattern in tutorials/lectures

Use of AI

- AI Use Statement

GenAI allowed to understand assignment
but what you submit must be in own words

- Key points:
 - Do you fully understand the submission?
 - Did you learn how to solve problems on your own?